DATA ANALYSIS PROJECT

PIZZA SALES NOMIC ANALYSIS

Using SQL

📌 Key Focus:
- Revenue Analysis
- Sales Trends
- Customer Behavior

Dataset:
- Orders
- Order Details
- Pizzas
- Pizza Types

DESCRIPTION:

- A comprehensive SQL-based analysis of pizza sales data to extract meaningful business insights.
It uncovers key trends in revenue, demand, and customer behavior across categories and time.
The project demonstrates strong data analysis skills for real-world decision-making.

Created By:

👤 Name: Aditya Shewale
Role: Data Analyst

SQL Project | 2026

CREATE DATABASE PIZZAS_PROJECTS;

USE PIZZAS_PROJECTS;

BASIC:

1. RETRIEVE TOTAL NUMBER OF ORDER PLACED.

SELECT COUNT(*) AS TOTAL_ORDERS FROM ORDERS;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TOTAL_ORDERS
21350

2. CALCULATE THE TOTAL REVENUE GENERATED FROM PIZZA SALES.

CREATE VIEW SALES_INFO AS

SELECT (OD.QUANTITY * P.PRICE) AS SALES

FROM ORDER_DETAILS OD

JOIN PIZZAS P ON OD.PIZZA_ID = P.PIZZA_ID;

SELECT CONCAT("$", ROUND(SUM(SALES),2)) AS TOTAL_SALES FROM SALES_INFO;

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TOTAL_SALES
$817860.05

3. IDENTIFY THE HIGHEST PRICED PIZZA

```
SELECT PT.NAME, MAX(P.price)
FROM PIZZA_TYPES PT
JOIN PIZZAS P ON PT.PIZZA_TYPE_ID = P.PIZZA_TYPE_ID
WHERE P.PRICE = (SELECT MAX(PRICE) FROM PIZZAS P);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

NAME	PRICE
The Greek Pizza	35.95

4. Identify the most common pizza size ordered.

```
SELECT p.size, SUM(od.quantity) AS total_quantity
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
GROUP BY p.size hcfORDER BY total_quantity DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

size	total_quantity
L	18956

5. List the top 5 most ordered pizza types along with their quantities.

```
SELECT pt.name AS pizza_name, SUM(od.quantity) AS total_quantity
FROM order_details od JOIN pizzas p  ON od.pizza_id = p.pizza_id
JOIN pizza_types pt  ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name ORDER BY total_quantity DESC LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

pizza_name	total_quantity
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

6. Join the necessary tables to find the total quantity of each pizza category ordered.

```
SELECT pt.category, SUM(od.quantity) AS total_quantity
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt  ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

7. Determine the distribution of orders by hour of the day.

```
SELECT EXTRACT(HOUR FROM time) AS order_hour,
 COUNT(order_id) AS total_orders GROUP BY order_hour
ORDER BY order_hour;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_hour	total_orders
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28

8. Join relevant tables to find the category-wise distribution of pizzas.

```
SELECT pt.category, SUM(od.quantity) AS total_quantity
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.category ORDER BY total_quantity DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_quantity
Classic	14888
Supreme	11987
Veggie	11649
Chicken	11050

9. Group the orders by date and calculate the average number of pizzas ordered per day.

```
SELECT AVG(daily_total) AS avg_pizzas_per_day
FROM ( SELECT  o.date, SUM(od.quantity) AS daily_total
FROM orders o JOIN order_details od ON o.order_id = od.order_id
   GROUP BY o.date) AS daily_orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_pizzas_per_day
138.4749

10. Determine the top 3 most ordered pizza types based on revenue.

```
SELECT  pt.name AS pizza_name SUM(od.quantity * p.price) AS total_revenue
FROM order_details od JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name ORDER BY total_revenue DESC LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

pizza_name	total_revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5

11. Calculate the percentage contribution of each pizza type to total revenue.

```
SELECT pt.name AS pizza_name, SUM(od.quantity * p.price) AS total_revenue,
   ROUND( SUM(od.quantity * p.price) * 100.0 /  SUM(SUM(od.quantity * p.price))
 OVER (),2) AS percentage_contribution FROM order_details od
JOIN pizzas p  ON od.pizza_id = p.pizza_id
JOIN pizza_types pt
   ON p.pizza_type_id = pt.pizza_type_id
GROUP BY pt.name
ORDER BY percentage_contribution DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

pizza_name	total_revenue	percentage_contribution
The Prosciutto and Arugula ...	24193.25	2.96
The Napolitana Pizza	24087	2.95
The Spinach and Feta Pizza	23271.25	2.85
The Big Meat Pizza	22968	2.81
The Pepperoni, Mushroom, ...	18834.5	2.3
The Chicken Alfredo Pizza	16900.25	2.07
The Chicken Pesto Pizza	16701.75	2.04
The Soppressata Pizza	16425.75	2.01
The Italian Vegetables Pizza	16019.25	1.96
The Calabrese Pizza	15934.25	1.95
The Spinach Pesto Pizza	15596	1.91
The Mediterranean Pizza	15360.5	1.88
The Spinach Supreme Pizza	15277.75	1.87
The Green Garden Pizza	13955.75	1.71
The Brie Carre Pizza	11588.4999999999	1.42

12. Analyze the cumulative revenue generated over time.

```
SELECT order_date, daily_revenue, SUM(daily_revenue) OVER (ORDER BY order_date)
AS cumulative_revenue FROM (SELECT o.date AS order_date, SUM(od.quantity * p.price)
AS daily_revenue FROM orders o
  JOIN order_details od
    ON o.order_id = od.order_id
  JOIN pizzas p
    ON od.pizza_id = p.pizza_id
  GROUP BY o.date) AS daily_sales ORDER BY order_date;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_date	daily_revenue	cumulative_revenue
2015-01-01	2713.8500000000004	2713.8500000000004
2015-01-02	2731.8999999999996	5445.75
2015-01-03	2662.4	8108.15
2015-01-04	1755.4500000000003	9863.6
2015-01-05	2065.95	11929.55
2015-01-06	2428.95	14358.5
2015-01-07	2202.2000000000003	16560.7
2015-01-08	2838.3499999999995	19399.05
2015-01-09	2127.3500000000004	21526.4
2015-01-10	2463.95	23990.350000000002
2015-01-11	1872.3000000000002	25862.65
2015-01-12	1919.0500000000002	27781.7
2015-01-13	2049.6000000000004	29831.300000000003
2015-01-14	2527.3999999999996	32358.700000000004
2015-01-15	1984.8000000000002	34343.50000000001

13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT category, pizza_name, total_revenue
FROM (SELECT pt.category, pt.name AS pizza_name, SUM(od.quantity * p.price)
 AS total_revenue, RANK() OVER (PARTITION BY pt.category ORDER BY SUM
(od.quantity * p.price) DESC) AS rank_in_category FROM order_details od
   JOIN pizzas p ON od.pizza_id = p.pizza_id
   JOIN pizza_types pt
      ON p.pizza_type_id = pt.pizza_type_id
   GROUP BY pt.category, pt.name
) ranked_pizzas
WHERE rank_in_category <= 3 ORDER BY category, total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	category	pizza_name	total_revenue
▶	Chicken	The Thai Chicken Pizza	43434.25
	Chicken	The Barbecue Chicken Pizza	42768
	Chicken	The California Chicken Pizza	41409.5
	Classic	The Classic Deluxe Pizza	38180.5
	Classic	The Hawaiian Pizza	32273.25
	Classic	The Pepperoni Pizza	30161.75
	Supreme	The Spicy Italian Pizza	34831.25
	Supreme	The Italian Supreme Pizza	33476.75
	Supreme	The Sicilian Pizza	30940.5
	Veggie	The Four Cheese Pizza	32265.70000000065
	Veggie	The Mexicana Pizza	26780.75
	Veggie	The Five Cheese Pizza	26066.5